



UDP编程初识

北京理工大学计算机学院
金旭亮

JDK中UDP相关的类型



DatagramSocket、DatagramPacket和MulticastSocket。



DatagramSocket负责发送与接收DatagramPacket。



MulticastSocket则负责发送与接收多播的DatagramPacket。

DatagramSocket

用于监听特定的端口，发送和接收UDP数据包。

构造函数：

- `DatagramSocket()`
- `DatagramSocket(int port)`
- `DatagramSocket(int port, InetAddress laddr)`
- `DatagramSocket(SocketAddress bindaddr)`



构造函数的参数，表示绑定的本机IP地址及端口，UDP在此进行监听。

使用UDP发送数据

```
public class SendUDPPacket {  
    2 usages  
    private static final String MESSAGE = "Hello,World!";  
  
    public static void main(String[] args) throws IOException {  
  
        byte[] buffer = MESSAGE.getBytes();  
        var address : InetSocketAddress = InetSocketAddress.getLocalHost();  
        var packet = new DatagramPacket(buffer, buffer.length, address, port: 1024);  
        try (var datagramSocket = new DatagramSocket()) {  
            datagramSocket.send(packet);  
            System.out.println("SENT: " + MESSAGE);  
        }  
    }  
}
```

DatagramPacket负责承载数据，注意，接收方应该监听1024端口，才能收到此数据包。



使用DatagramSocket发送DatagramPacket。

使用UDP接收数据

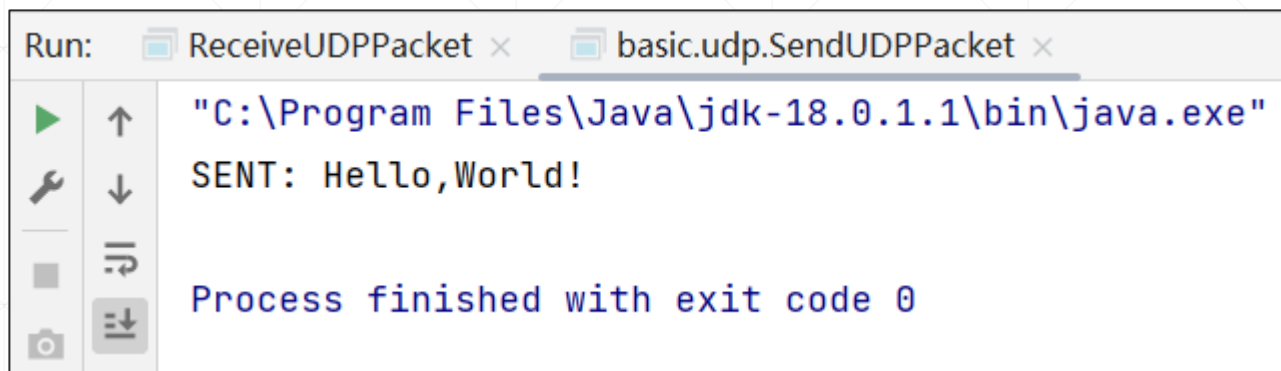
接收方的DatagramSocket在1024端口监听。

```
public class ReceiveUDPPacket {  
    1 usage  
    private static final int BUFLEN="Hello,World!".getBytes().length;  
    public static void main(String[] args) throws IOException {  
        try(var socket=new DatagramSocket( port: 1024)){  
            var receiveData=new byte[BUFLEN];  
            var packet=new DatagramPacket(receiveData,receiveData.length);  
            socket.receive(packet);  
            var message=new String(packet.getData());  
            System.out.println("RECEIVED:"+message);  
        }  
    }  
}
```

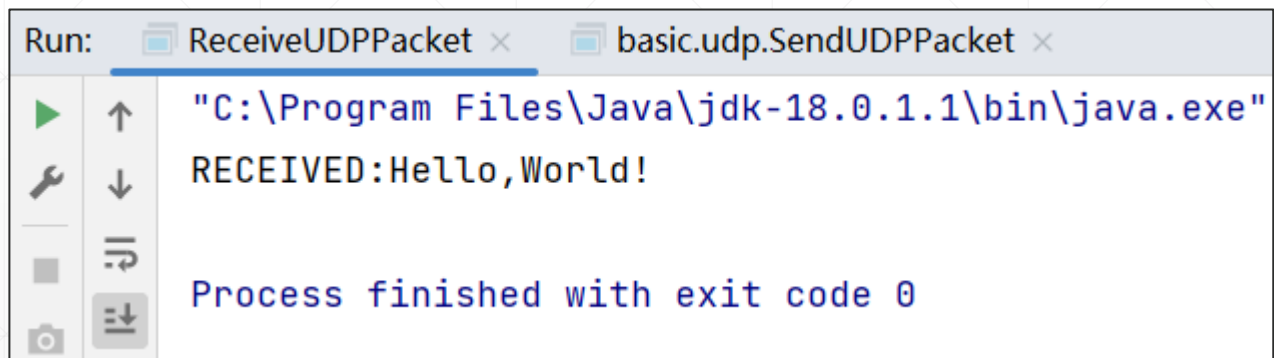


使用DatagramPacket存储收到的消息。

运行截图



```
Run: ReceiveUDPPacket x basic.udp.SendUDPPacket x
"C:\Program Files\Java\jdk-18.0.1.1\bin\java.exe"
SENT: Hello,World!
Process finished with exit code 0
```



```
Run: ReceiveUDPPacket x basic.udp.SendUDPPacket x
"C:\Program Files\Java\jdk-18.0.1.1\bin\java.exe"
RECEIVED:Hello,World!
Process finished with exit code 0
```

要点

- (1) 端口一致
- (2) 数据发送与接收缓冲区大小要一致
- (3) 数据发送的编码要一致

有“回执”的UDP信息交换方式



```
Run: UDPReceiver x sendcallback.UDPSender x
"C:\Program Files\Java\jdk-17\bin\java.exe"
消息已经发送到目标机器的端口: 10001
等待回信...
收到回执: 你的信息我已经收到, 勿念!
回信方信息: ip:192.168.0.109 port:10001
Process finished with exit code 0
```

发信方

```
Run: UDPReceiver x sendcallback.UDPSender x
"C:\Program Files\Java\jdk-17\bin\java.exe"
正在监听: 10001
收到信息: 这是一个测试用字符串
发送方信息: ip:192.168.0.109 port:54508
发送回执
回执已发送...
Process finished with exit code 0
```

收信方

定义常量

```
public class UDPConstants {  
    2 usages  
    public static int SENDER_LISTEN_PORT=10000;  
    4 usages  
    public static int RECEIVER_LISTEN_PORT=10001;  
}
```

以上常量定义了发送方与接收方监听的端口。

接收方

接收方接收到消息之后，从消息中提取出发送方的地址信息，以此为参数，创建一个新的DatagramPacket，再发给发送方。

```
public class UDPReceiver {  
    public static void main(String[] args) throws IOException {  
        System.out.println("正在监听: " + UDPConstants.RECEIVER_LISTEN_PORT);  
        try(var ds = new DatagramSocket(UDPConstants.RECEIVER_LISTEN_PORT)){  
            var dataBuffer = new byte[1024];  
            var receivePack = new DatagramPacket(dataBuffer, dataBuffer.length);  
            //在此阻塞等待接收  
            ds.receive(receivePack);  
            // 打印接收到的信息与发送者的信息  
            // 发送者的IP地址  
            String ip = receivePack.getAddress().getHostAddress();  
            int port = receivePack.getPort(); // 发送者的端口  
            int dataLen = receivePack.getLength(); //获取接收到的数据长度  
            //转换为字符串  
            String message = new String(receivePack.getData(), offset: 0, dataLen);  
            System.out.println("收到信息: " + message);  
            System.out.print("发送方信息: ");  
            System.out.println("ip:" + ip + "\tport:" + port);  
            System.out.println("发送回执");  
            //生成回执信息  
            var callbackMessage = "你的信息我已经收到, 勿念!".getBytes(StandardCharsets.UTF_8);  
            var sendPack = new DatagramPacket(callbackMessage, callbackMessage.length,  
                receivePack.getAddress(), UDPConstants.SENDER_LISTEN_PORT);  
            ds.send(sendPack); //发送回执  
            System.out.println("回执已发送...");  
        }  
    }  
}
```

发送方

```
public class UDPSender {  
    public static void main(String[] args)  
        throws IOException {  
        sendMessage("这是一个测试用字符串");  
        System.out.println("等待回信...");  
        waitCallback();  
    }  
}
```

先发送信息，再等待回执

```
private static void sendMessage(String message) throws IOException {  
    byte[] messageData = message.getBytes(StandardCharsets.UTF_8);  
    //目标地址是必须设置的，否则，会抛出异常: IllegalArgumentException  
    var address = InetAddress.getLocalHost();  
    //也可以设置为广播地址 address=InetAddress.getByname("255.255.255.255");  
    var sendPacket = new DatagramPacket(messageData, messageData.length, address,  
        UDPConstants.RECEIVER_LISTEN_PORT);  
    try(var datagramSocket = new DatagramSocket()){  
        datagramSocket.send(sendPacket);  
        System.out.println("消息已经发送到目标机器的端口: " + UDPConstants.RECEIVER_LISTEN_PORT);  
    }  
}
```

```
private static void waitCallback() throws IOException {  
    try (var ds = new DatagramSocket(UDPConstants.SENDER_LISTEN_PORT)) {  
        var buf = new byte[1024];  
        DatagramPacket receivePack = new DatagramPacket(buf, buf.length);  
        // 接收  
        ds.receive(receivePack);  
        String ip = receivePack.getAddress().getHostAddress();  
        int port = receivePack.getPort();  
        int dataLen = receivePack.getLength();  
        String callback = new String(receivePack.getData(), offset: 0, dataLen);  
        System.out.println("收到回执: " + callback);  
        System.out.print("回信方信息: ");  
        System.out.println("ip:" + ip + "\tport:" + port);  
    }  
}
```

练习



掌握了UDP回送信息的技巧，我们就可以编写出实现以下“自动发现”功能的程序：

某客户端启动之后，发送一个UDP广播消息，某服务器收到之后，发回一个响应消息，放入本机的IP地址和开放的端口号，这样，客户端就能“发现”服务端，并且能使用TCP协议与之通讯了。